- Implement techniques like face alignment to improve recognition reliability.

5. User-Friendly Features:
   - Add face labeling to display the names of recognized individuals on the video feed.
   - Include an attendance tracking module that logs the entry and exit times of individuals.
   - Provide a GUI or web interface for ease of use and interaction.

6. Testing and Deployment:
   - Test the system under various real-world conditions, including crowded environments and diverse lighting scenarios.
   - Deploy the application on different platforms, such as PCs, Raspberry Pi, or cloud-based servers, depending on use cases.

Outcome:

A robust real-time face recognition system that can be used for applications in security, attendance tracking, and more. This project demonstrates the practical use of computer vision and deep learning in creating intelligent, real-time systems.

Example Code:

```
import cv2
import numpy as np
import face_recognition
from datetime import datetime

# Step 1: Load Known Faces and Encodings known_face_encodings = []
known_face_names = []

# Load sample images and encode faces
known_image = face_recognition.load_image_file('person1.jpg') known_encoding =
face_recognition.face_encodings(known_image)[0]
known_face_encodings.append(known_encoding) known_face_names.append('Person 1')

# Step 2: Initialize Webcam
video_capture = cv2.VideoCapture(0)

# Step 3: Process Each Frame
```